

Employee Salary Management System — Requirements

Goal

Replace ACME's spreadsheet-based salary tracking with a web application that lets an **HR Manager** maintain accurate, current, and historical salary data for ~10,000 employees across multiple countries, and answer questions about how the org pays its people (e.g. pay by department/country, headcount cost, pay changes over time) without exporting data to Excel first.

Persona

HR Manager — trusted back-office user who creates/updates employee salary records, uploads bulk data during migration, and pulls reports/insights for leadership. Not building for employee self-service or payroll disbursement in this phase.

Scope & Features (MVP)

1. **Employee & compensation records** — CRUD for employee profile (name, employee ID, department, country, currency) and compensation (base salary, effective date, pay frequency). Salary is stored as `(amount, currency)` — never converted destructively.
2. **Salary history (append-only)** — every change creates a new dated record rather than overwriting the old one. This is the backbone of "how does the org pay people over time" and doubles as an audit trail (who changed what, when).
3. **Bulk import from CSV/Excel** — ACME's current source of truth is spreadsheets; a one-time and repeatable import path is required to make migration realistic, with row-level validation and an error report (not silent partial failure).
4. **Search & filter** — by name/ID, department, country, salary band.
5. **Reporting/insights** — aggregate views the HR Manager actually asks for: headcount and total pay cost by department/country, average/median salary by department, and salary distribution/bands. Delivered as structured dashboard queries, not free-text NLP — see "Out of scope" below.
6. **Authentication** — a logged-in HR Manager session gates all access to salary data, since compensation data is highly sensitive.
7. **Seed script (10,000 employees)** — a deterministic, re-runnable script that generates realistic synthetic data (spread across departments/countries/currencies, with a salary history of more than one entry per employee) so the system can be exercised at target scale from day one, instead of scale problems surfacing only near submission. Doubles as a stand-in for the "bulk import" path during development and as fixtures for performance-sensitive tests (pagination, aggregate queries).

Non-functional priorities

- Correctness and auditability of salary data over raw feature count.

- **Designed to extend, not to be rewritten** — this is phase 1 of a longer roadmap (payroll, self-service, RBAC, live FX are all foreseeable next phases per the "out of scope" table below). The architecture should let those be *added*, not force a redesign. Concrete hooks for this are listed under "Built for Phase 2" below.
- Must comfortably handle 10,000 employees × multi-year salary history (pagination, indexed queries) — this is a "boring, correct at scale" system, not a prototype. The seed script exists so this is validated against real volume, not assumed.
- API and data model designed so payroll/self-service could be added later without a data-model rewrite.

Built for Phase 2 (concrete hooks, not just intent)

Each "out of scope" item above has a specific reason it won't require a rewrite later:

- **Layered architecture** (route → service → repository). Business rules live in the service layer, not in route handlers or SQL — a new consumer (payroll job, self-service API) can reuse the same service functions instead of duplicating logic.
- **One write path for employees.** Manual CRUD, bulk import, and the seed script all go through the same "create/update employee" service function, so validation and history-writing logic exist in exactly one place, however data enters the system.
- **(amount, currency) kept intact, conversion isolated behind one function.** Swapping a static rate table for a live FX API later is a one-function change, not a schema change.
- **Auth checks centralized in middleware, single (HR_MANAGER) role today.** Adding roles (e.g. (EMPLOYEE) for self-service) is additive — new checks, not a rewrite of existing ones.
- **Append-only salary history is already the audit log.** A future payroll module or notification system (e.g. "notify on raise") can subscribe to the same history writes instead of a bolted-on separate audit table.
- **API versioned from day one ((/api/v1/...)),** so phase-2 breaking changes don't break existing integrations.
- **Stateless API (token-based auth, no server-side session state).** Scaling to more employees/traffic is a deployment change (add instances behind a load balancer), not a code change.
- **Reporting built as reusable aggregate queries,** not baked into a UI component — a future NL/chatbot layer (explicitly deferred above) would call the same queries rather than needing new aggregation logic.

Deliberately out of scope (and why)

Excluded	Reasoning
Payroll processing (tax withholding, statutory deductions, bank disbursement)	Distinct problem domain with per-country legal/compliance requirements; conflating it with salary <i>record-keeping</i> would blow up scope for an assessment and is a different system of record in most real orgs.
Employee self-service portal (payslips, personal view)	Different persona than the one specified (HR Manager only). Adding it doubles the auth/permission surface for no assessment value.
Multi-role RBAC (admin/manager/employee tiers)	Only one persona was given. I model a single <code>HR_MANAGER</code> role now but design auth so roles can be added later, rather than building permission infrastructure nobody asked for.
Live FX conversion via 3rd-party API	Introduces an external dependency/rate-limit/failure mode for a "nice to have" (cross-currency roll-ups). MVP keeps <code>(amount, currency)</code> intact and does comparisons within a currency/country first; a static/periodic rate table is a natural fast-follow.
Free-text/NLP "ask questions" interface	The problem statement says HR should be able to "answer questions about pay" — I'm interpreting that as good aggregate reporting, not a chatbot. An LLM-backed query layer is an interesting extension but adds non-determinism to a system whose core value is trustworthy numbers; I'd rather ship correct dashboards than a flashy but unreliable NL layer.
Notifications/email, org chart, leave/performance management	Out of the salary domain entirely.
Mobile app / native clients	"Web-based software" was explicit in the ask.

Assumptions

Stated explicitly so they're falsifiable, not silently baked in:

- "Salary" means **annual gross base pay**, before tax/deductions — this is a record-keeping system, not a payroll calculator (see out-of-scope).
- Each employee has exactly **one currency at a time**, generally their country's local currency; a currency change is itself a `SalaryRecord` (reason: `CURRENCY_CHANGE`), not a silent conversion of history.

- **Department is a flat, fixed list** for phase 1 (e.g. Engineering, Sales, HR, Finance...), not a hierarchical org chart — org-chart modeling is a separate concern from compensation.
- **Country is the employee's work location**, used for filtering/reporting and as the default currency signal — not a proxy for legal employment entity or tax jurisdiction.
- One HR Manager account is trusted with full access to all 10,000 records; per-country or per-department access restriction is a Phase 2 RBAC concern (already flagged above).

Success looks like

An HR Manager can: import the existing employee base in one pass with clear validation errors, update an individual's salary and see the prior value preserved in history, and answer "what's our average salary in Engineering in Germany" in a few clicks — all backed by tests that pin down the behavior above (TDD).